

Ricorsione

Fattoriale

```
#include <stdio.h>

int factorial(int n)
{
    // Calculating factorial of number
    if (n == 0)
        return 1;
    return n * factorial(n - 1);
}

int factorialInner(int n, int temp) {
    if (n==0)
        return temp;
    return factorialInner(n-1, temp*n);
}
int factorialTail(int n) {
    return factorialInner(n, 1);
}

int main()
{
    int num = 5;
    printf("%d\n", factorial(num));
    printf("%d\n", factorialTail(num));
    return 0;
}
```

Dec2Bin

```
int decToBin(int d)
{
    if (d == 0)
        return 0;
    return (d % 2 + 10 * decToBin(d / 2));
}
int main(void) {
    int d = 11;
    printf("%d\n", decToBin(d));
    return 0;
}
```

Palindroma

```

bool isPalindromeRec(int s[], int left, int right) {
    if (left >= right)
        return true;
    if (s[left] != s[right])
        return false;
    return isPalindromeRec(s, left + 1, right - 1);
}

bool isPalindrome(int s[], int len) {
    return isPalindromeRec(s, 0, len - 1);
}

int main(void) {
    int s[] = {'a', 'b', 'c', 'b', 'a', 'd'};

    printf ("%d", isPalindrome(s, sizeof(s) / sizeof(s[0])));
    return 0;
}

```

Torri di Hanoi

Il gioco consiste nello spostare i dischi (che possono essere in numero variabile) dalla colonnina, dove sono impilati dal più grande al più piccolo, in una delle altre due seguendo le seguenti regole:

- si deve spostare un disco alla volta
- non si può mai spostare un disco più grande sopra uno più piccolo

Realizzare una animazione della successione di mosse.

Per il rendering utilizzare la tecnica di stampare una linea alla volta costruendo una sequenza di caratteri con printf successive

```

#include <stdio.h>
#define N 3
#define NMPGS 3
void printSegm(int n, char c){
    for(int i=0; i< n; i++)
        printf("%c", c);
}
void printDiscOnPeg(int disc){
    printSegm(N-disc, ' '); //vuoto
    printSegm(disc, '*'); //piena
    printf("|"); //piolo
    printSegm(disc, '*'); //piena
    printSegm(N-disc, ' '); //vuoto
}
void display(int mx[3][N]){
    for(int i=N-1; i>=0; i--){ //senso verticale dall'alto
        for(int j=0; j<NMPGS; j++){ //per tutti i pioli
            printDiscOnPeg(mx[j][i]);
        }
    }
}

```

```

        printf("\n");
    }
    printf("\n");
}
//area globale
int pegs[NMPGS][N];
int indexs[NMPGS]; //indice del top del piolo

void move(int from, int to){
    printf("f%d t%d\n", from, to);
    pegs[to][++indexs[to]] = pegs[from][indexs[from]];
    pegs[from][indexs[from]--] = 0;
    display(pegs);
}

void towerOfHanoi(int n, int from, int to, int aux)
{
    if (n == 0)
        return;
    towerOfHanoi(n-1, from, aux, to);
    move( from, to);
    towerOfHanoi(n-1, aux, to, from);
}

int callHanoi(void) {
    for (int i=0; i<N;i++)
        pegs[0][i] = N-i;
    indexs[0] = N-1;
    indexs[1] = -1; //piolo vuoto
    indexs[2] = -1; //piolo vuoto
    display(pegs);
    towerOfHanoi(N, 0, 2, 1);
    return 0;
}

```